

Reviewer Pack — Minimal Number of Automorphism Group Orbits and Resolution P...

Entry 42b1f61ef4d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 13:31:38 UTC

Minimal Number of Automorphism Group Orbits and Resolution Proof Width Inequality

Entry ID: 42b1f61ef4d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 13:31:38 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Automorphism Groups) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every Boolean satisfiability instance φ with n variables, the minimal number of orbits of automorphisms of φ 's associated graph G is linearly correlated with its resolution proof width $w(\varphi)$, such that the number of orbits $|\text{Orbits}(G)| = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

The study of automorphism groups in group theory can provide insights into the symmetry properties of computational problems. By linking this to resolution proof complexity, we may uncover new structures that influence the complexity of solving satisfiability problems, potentially revealing new complexities or separations.

Taxonomy category: AutomorphismGroupOrbit (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6fcab90139cc3007

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 24 out of 30 random seeds, the correlation coefficient between the number of orbits $|\text{Orbits}(G)|$ and resolution proof width $w(\varphi)$ exceeds 0.9, and all computed values of $|\text{Orbits}(G)|$ are within a range of $\pm 20\%$ around the median value of $|\text{Orbits}(G)|$ for that seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "automorphism group orbits" AND "resolution proof width" AND boolean satisfiability" - "group theory" AND automorphism* AND "Boolean Satisfiability" AND resolution* - "resolution proof complexity" AND automorphism group* AND boolean logic

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=122.7s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for i in range(1, n+1):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def is_satisfiable(cnf):
        stack = []
        assignment = {}

        def dpll():
            if not cnf:
                return True
            literal = find_unassigned_variable(cnf)
            if literal is None:
                return False

            for value in [True, False]:
                new_assignment[literal] = value
                new_cnf = propagate(cnf, literal, value)
                if dpll():
                    return True
                del new_assignment[literal]

            return False

    return {"cnf": cnf, "satisfiable": is_satisfiable(cnf)}
```

```

def find_unassigned_variable(cnf):
    for clause in cnf:
        for literal in clause:
            if literal not in new_assignment and -literal not in new_assignment:
                return literal
    return None

def propagate(cnf, literal, value):
    new_cnf = []
    for clause in cnf:
        if literal in clause:
            continue
        if -literal in clause:
            clause.remove(-literal)
            if not clause:
                return []
            new_cnf.append(clause)
        else:
            new_cnf.append(clause)
    return new_cnf

return dpll()

def calculate_orbits(cnf):
    graph = {}
    for clause in cnf:
        for literal1 in clause:
            for literal2 in clause:
                if literal1 != literal2:
                    if literal1 not in graph:
                        graph[literal1] = set()
                    if literal2 not in graph:
                        graph[literal2] = set()
                    graph[literal1].add(literal2)
                    graph[literal2].add(literal1)

def dfs(node, visited):
    stack = [node]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                stack.append(neighbor)

orbits = 0
visited = set()
for literal in graph:
    if literal not in visited:
        dfs(literal, visited)
        orbits += 1

return orbits

def calculate_resolution_width(cnf):
    clauses = [set(clause) for clause in cnf]

```

```

queue = clauses.copy()
while queue:
    clause1 = queue.pop(0)
    for clause2 in queue:
        new_clause = set()
        for literal in clause1:
            if -literal not in clause2:
                new_clause.add(literal)
        for literal in clause2:
            if -literal not in clause1:
                new_clause.add(literal)
        if len(new_clause) == 0:
            return 0
        queue.append(new_clause)
    return max(len(clause) for clause in clauses)

n = random.randint(5, 40)
cnf = generate_cnf(n)
orbits = calculate_orbits(cnf)
width = calculate_resolution_width(cnf)

return {
    "metric_name": "Orbit Width Ratio",
    "metric_value": Fraction(orbits, width) if width != 0 else None,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": orbits == width,
    "counterexample": "" if orbits == width else f"Orbits: {orbits}, Width: {width}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_count = sum(1 for r in results if r["conjecture_holds"])

    mean = sum(metric_values) / len(metric_values) if metric_values else 0
    std = (sum((x - mean) ** 2 for x in metric_values) / len(metric_values)) ** 0.5 if metric_values else 0
    support_fraction = support_count / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TRIAL: {'metric_name': 'Orbit Width Ratio', 'metric_value': None, 'instances_tested': 1, 'n_max': 33,

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete the required number of trials to evaluate the conjecture. | next: Re-run the test with a sufficient number of instances and ensure that it completes without crashing or timing out.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13895
2	preregistration	ollama_remote	glm4:latest	0	0	9682
3	novelty	ollama_remote	glm4:latest	0	0	8402
4	novelty	ollama_remote	glm4:latest	0	0	9200
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52448
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19198
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15029
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13496
9	judge	ollama_remote	glm4:latest	0	0	82197

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 223547 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/42b1f61ef4d3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/42b1f61ef4d3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/42b1f61ef4d3.tar.gz` (if generated)